



Lesson Plan

Functions

Q. What are functions ?

A function is a block of code which only runs when it is called.

You can pass data, known as parameters, into a function.

Functions are used to perform certain actions, and they are important for reusing code: Define the code once, and use it many times.

Q. Why functions ?

Because using functions makes the code neat and reusable.

For example , below written code is to write a function to sum two numbers.

```
#include<iostream>
using namespace std;
int sum(int a,int b){
    return a + b;
}
int main(){
    cout<<sum(4 , 5);
}
```

Output : 9

Explanation : In this code, we have written a separate function named as sum , that accepts two parameters a and b and returns an integer, which is the sum of the passed parameters.

Components of a function :

There are mainly 3 components of any function :

Function name

Function return type

Argument list (Optional)

Function name : This is name of the function that the user chooses. In general practice, function name does not start with uppercase character.

For example, myCode , difference are valid function names.

Function return type : This defines what type of data will be returned by the function. It can be int , char , string , void etc.

Argument list : This is a list of parameters that we will be passing in our function. There can be any number of parameters in a function. A function can also be created without any argument list therefore this is optional.

A C++ function consist of two parts:

Declaration: The return type, the name of the function, and parameters (if any)

Definition: The body of the function (code to be executed)

Note : A function must be declared before it is called. This will become more clear with the help of the below given example :

```
int main() {
    functionWithPW();
    return 0;
}
void functionWithPW() {
    cout << "Hello pwians";
}
// Error
```

The above stated code will generate an error, since functionWithPW is not declared before its call. To avoid this error we can split the definition and declaration of the function in two different parts as shown in the example below:

```
//function declaration
void functionWithPW();
//main program
int main() {
    functionWithPW();
    return 0;
}
//function definition
void functionWithPW() {
    cout << "Hello pwians";
}
```

Different types of passing a parameter :

A parameter can be passed primarily in two ways :

Pass by value

Pass by reference

Pass by value : This method copies the actual value of an argument into the formal parameter of the function. In this case, changes made to the parameter inside the function have no effect on the argument.

For example :

```
#include<iostream>
using namespace std;
void swap(int a,int b){
    int temp = a;
    a = b;
    b = temp;
    return;
}
int main(){
    int a = 6 , b = 8;
```

```
cout<<"Values of a and b respectively before swap : "<<a<<"
"<<b<<endl;
swap(a,b);
cout<<"Values of a and b respectively after swap : "<<a<<"
"<<b<<endl;
}
```

Output :

Values of a and b respectively before swap : 6 8

Values of a and b respectively after swap : 6 8

Here we can see the values have not been swapped. This is because we have passed the parameters by value that means a copy of a and b have been passed and the changes made by the swap function are observed in those copies and not reflected back in the original variables. This is pass by value.

Pass by reference : This method copies the reference of an argument into the formal parameter. Inside the function, the reference is used to access the actual argument used in the call. This means that changes made to the parameter affect the argument.

For example,

```
#include<iostream>
using namespace std;
void swap(int &a,int &b){
int temp = a;
a = b;
b = temp;
return;
}
int main(){
int a = 6 , b = 8;
cout<<"Values of a and b respectively before swap : "<<a<<"
"<<b<<endl;
swap(a,b);
cout<<"Values of a and b respectively after swap : "<<a<<"
"<<b<<endl;
}
```

Output :

Values of a and b respectively before swap : 6 8

Values of a and b respectively after swap : 8 6

Here, we have passed the address of the original variables a and b . Therefore all the changes made by the swap function are reflected in the original variables as well.

Formal Parameter : A variable and its type as they appear in the prototype of the function or method.

Actual Parameter : The variable or expression corresponding to a formal parameter that appears in the function or method call in the calling environment.

Formal parameters Vs Actual parameters

Actual Parameters	Formal Parameters
Defined when it is invoked.	Defined when the function is called.
Passed on calling a function	They will be in the called function.
Data types will not be mentioned.	The data type must be included.
They are written in the function call.	They are written in the function definition.
They may be constant values.	They can be local variables.

Scope of a variable : scope of a variable is defined as the extent of the program code within which the variable can be accessed or declared or worked with.

Variables are of two types :

Local variables : Variables defined within a function or block are said to be local to those functions. These variables are not accessible outside the function or code block.

For example,

```
#include<iostream>
using namespace std;
void function1(){
int a = 5 ;
}
void myFunction(){
cout<<a<<" ";
}
int main(){
function1();
myFunction();
}
```

Output :

error: 'a' was not declared in this scope.

Global variables : They can be accessed from any part of the program. They are available through out the life time of a program. They are declared at the top of the program outside all of the functions or blocks.

For example,

```
#include<iostream>
using namespace std;
```

```
int b = 10;
void myFunction(){
cout<<b<<endl;
}
int main(){
myFunction();
cout<<b<<endl;
}
```

Output:

10
10

Q1. Pascal triangle:

```
#include <iostream>

// Function to calculate and print Pascal's Triangle
void printPascalsTriangle(int n) {
    for (int i = 0; i < n; i++) {
        int coefficient = 1;

        // Print spaces for alignment
        for (int space = 1; space ≤ n - i; space++)
            std::cout << " ";

        for (int j = 0; j ≤ i; j++) {
            // Print the current coefficient
            std::cout << coefficient << " ";

            // Update the coefficient for the next iteration
            coefficient = coefficient * (i - j) / (j + 1);
        }

        std::cout << std::endl;
    }
}

int main() {
    int numRows;

    // Input the number of rows for Pascal's Triangle
    std::cout << "Enter the number of rows for Pascal's Triangle: ";
    std::cin >> numRows;

    // Print Pascal's Triangle
    printPascalsTriangle(numRows);

    return 0;
}
```

Q2. Write a function to compute the greatest common divisor of two given numbers

```
#include <iostream>

// Function to compute the greatest common divisor (GCD) using
Euclidean algorithm
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int main() {
    int num1, num2;

    // Input two numbers
    std::cout << "Enter the first number: ";
    std::cin >> num1;

    std::cout << "Enter the second number: ";
    std::cin >> num2;

    // Compute and print the GCD
    int result = gcd(num1, num2);
    std::cout << "Greatest Common Divisor (GCD) of " << num1 << "
and " << num2 << " is: " << result << std::endl;

    return 0;
}
```

Q3. Print the factorials of first n numbers

```
#include <iostream>

// Function to calculate the factorial of a number without
recursion
unsigned long long factorial(int n) {
    unsigned long long result = 1;
    for (int i = 2; i ≤ n; ++i) {
        result *= i;
    }
    return result;
}

int main() {
    int n;
```

```
// Input the value of n
std::cout << "Enter the value of n: ";
std::cin >> n;

// Print the factorials of the first n numbers
std::cout << "Factorials of the first " << n << " numbers:"
<< std::endl;
for (int i = 0; i ≤ n; ++i) {
    std::cout << "Factorial of " << i << ": " << factorial(i)
<< std::endl;
}

return 0;
}
```

Q4. Swap two numbers

```
#include <iostream>

// Function to swap two numbers
void swapNumbers(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int num1, num2;

    // Input two numbers
    std::cout << "Enter the first number: ";
    std::cin >> num1;

    std::cout << "Enter the second number: ";
    std::cin >> num2;

    // Print the numbers before swapping
    std::cout << "Before swapping: num1 = " << num1 << ", num2 = "
<< num2 << std::endl;

    // Call the swap function
    swapNumbers(num1, num2);

    // Print the numbers after swappin
```




**THANK
YOU !**

